Git Cheat Sheet

The commands you actually use — one page, printable, pinnable

PeakPost
v1.0

SETUP

`git config --global user.name "Name"`
Set identity (once per machine)

`git config --global user.email "..."`
Email on every commit

`git config --global init.defaultBranch main`
Use "main" for new repos

`git config --global pull.rebase true`
Rebase on pull (cleaner history)

START A REPO

`git init`
New repo in current folder

`git clone <url>`
Copy a remote repo locally

`git clone --depth 1 <url>`
Shallow clone (faster, smaller)

DAILY FLOW

`git status -s`
Short status — run constantly

`git add .`
Stage all changes

`git add -p`
Stage interactively by chunk

`git commit -m "feat: ..."`
Commit staged changes

`git commit -am "..."`
Stage + commit tracked files

`git commit --amend --no-edit`
Add to last commit (not pushed yet!)

BRANCHING

`git branch`
List local branches

`git switch -c feature/x`
Create and switch to new branch

`git switch main`
Switch to existing branch

`git switch -`
Switch to previous branch

`git branch -d old-branch`
Delete merged branch

`git push origin --delete branch`
Delete remote branch

REMOTES

`git remote -v`
List remotes

`git fetch --all --prune`
Refresh state, clean dead branches

`git pull --rebase`
Get latest + replay your commits

`git push -u origin branch`
Push new branch, set upstream

`git push --force-with-lease`
SAFE force push (never plain --force)

MERGING

`git merge feature/x`
Merge branch into current

`git merge --no-ff feature/x`
Preserve branch history as merge bubble

`git merge --squash feature/x`
Squash all commits into one

`git merge --abort`
Back out if things get weird

REBASING

`git rebase main`
Replay your commits on top of main

`git rebase -i HEAD~5`
Interactive: squash, reorder, drop

`git rebase --continue`
Resume after resolving conflicts

`git rebase --abort`
Escape hatch

VIEWING HISTORY

`git log --oneline`
Compact log

`git log --oneline --graph --all`
Full tree view

`git log --author="Jane"`
Filter by author

`git log -S "text"`
Find commits that added/removed text

`git blame file.js`
Who changed each line?

`git show <hash>`
Show a specific commit

DIFFING

`git diff`
Unstaged changes

`git diff --staged`
Staged changes

`git diff main..feature`
Differences between branches

`git diff --name-only`
Just list changed files

`git diff --stat`
Summary of changed files

STASHING

`git stash`
Park changes temporarily

`git stash -u`
Include untracked files

`git stash list`
See parked changes

`git stash pop`
Re-apply and remove top stash

`git stash apply stash@{2}`
Apply a specific stash

UNDOING

`git restore file.js`
Discard unstaged changes to file

`git restore --staged file.js`
Unstage (keep changes)

`git reset --soft HEAD~1`
Undo commit, keep staged

`git reset --hard HEAD~1`
DESTRUCTIVE — nuke last commit

`git revert <hash>`
Safe undo (creates a new commit)

TAGS

`git tag`
List tags

`git tag -a v1.0.0 -m "Release"`
Annotated tag (for releases)

`git push origin v1.0.0`
Push one tag

`git push --tags`
Push all tags

`git tag -d v1.0.0`
Delete local tag

CHERRY-PICK

`git cherry-pick <hash>`
Apply a commit from anywhere

`git cherry-pick a..b`
Pick a range

`git cherry-pick --continue`
Resume after conflict

`git cherry-pick --abort`
Bail out

FIND A BUG (BISECT)

`git bisect start`
Begin binary search

`git bisect bad`
Mark current as broken

`git bisect good v1.0`
Mark tag as working

`git bisect run npm test`
Automate with a test command

`git bisect reset`
End session

EMERGENCY RECOVERY

`git reflog`
Your personal undo history (90 days)

`git reset --hard HEAD@{3}`
Restore to a reflog point

`git branch recovered <hash>`
Resurrect a deleted branch

`git fsck --full`
Diagnose repo problems

`git <op> --abort`
ANY multi-step op can be aborted

CLEANUP

`git clean -n`
Preview untracked files to remove

`git clean -fd`
Delete untracked files + dirs

`git branch --merged main`
Safe-to-delete branches

`git gc`
Garbage collect (smaller .git)

CONVENTIONAL COMMITS

`feat: new feature`

`fix: bug fix`

`docs: documentation`

`refactor: restructuring`

`perf: performance`

`test: tests`

`chore: tooling, deps`

`feat!: breaking change`

SAFETY RULES

Never `git push --force` to shared branches.

Always `git pull --rebase` before you push.

Don't amend commits you've already pushed.

Commit often. Undoing a small change is easy.

When in doubt: `git status` & `git reflog` .